15 Conexión a BDs

Uso de base de datos con Python

En este capítulo vamos a hacer un breve introducción al uso de base de datos con Python, para lo que utilizaremos SQLite, una base de datos sencilla pero potente, que Python ya trae incluida.

- SQL es un lenguaje estándar para gestionar bases de datos relacionales, permitiendo operaciones como crear, consultar, actualizar y eliminar datos.
- SQLite es una base de datos ligera, sin servidor, ideal para aplicaciones pequeñas, prototipos y almacenamiento local.
- Python incluye el módulo sqlite3 para interactuar con SQLite, facilitando operaciones CRUD (Crear, Leer, Actualizar, Eliminar).
- DB Browser for SQLite es una herramienta gráfica que simplifica la visualización y gestión de bases de datos SQLite sin necesidad de comandos SQL complejos.
- Vamos a proporcionar una introducción práctica a SQL, SQLite con Python y el uso de DB Browser for SQLite, con ejemplos claros.

Introducción a SQL

SQL (Structured Query Language) es un lenguaje diseñado para interactuar con bases de datos relacionales, donde los datos se organizan en tablas, con filas y columnas. Es ampliamente utilizado para gestionar datos en aplicaciones, desde pequeñas apps móviles, hasta sistemas empresariales. SQL permite realizar tareas como consultar datos, insertar nuevos registros, actualizar información existente y eliminar datos, entre otras.

Uso de SQLite con Python

Python incluye el módulo sqlite3, que permite conectar, crear y manipular bases de datos SQLite. Puedes crear tablas, insertar datos, realizar consultas y gestionar transacciones de manera sencilla. Este módulo es parte de la biblioteca estándar de Python, por lo que no requiere instalación adicional.

Uso de DB Browser for SQLite

DB Browser for SQLite es una herramienta gráfica de código abierto que permite visualizar y editar bases de datos SQLite. Con una interfaz similar a una hoja de cálculo, facilita la creación de tablas, la inserción de datos y la ejecución de consultas SQL, siendo ideal tanto para principiantes como para desarrolladores.

15 Conexión a BDs

¿Qué es una Base de Datos Relacional?

Antes de profundizar en SQL, es importante entender el modelo que soporta: las bases de datos relacionales. Este modelo, propuesto por Edgar F. Codd en 1970, organiza los datos en tablas (similares a hojas de cálculo), donde cada tabla representa una entidad (por ejemplo, "Clientes" o "Productos").

Componentes clave:

- **Tabla (Table):** Una colección de datos organizados en filas y columnas. Cada tabla tiene un nombre único.
- **Fila (Row o Tuple):** Representa un registro individual, como un cliente específico.
- **Columna (Column o Attribute):** Define un atributo de los datos, como "Nombre", "Edad" o "ID". Cada columna tiene un tipo de dato (e.g., INTEGER, TEXT, REAL en SQLite).
- **Clave Primaria (Primary Key):** Un identificador único para cada fila, que evita duplicados (e.g., un ID autoincremental).
- **Clave Externa (Foreign Key):** Un campo que enlaza una tabla con otra, asegurando la integridad referencial (e.g., el ID de un cliente en una tabla de pedidos).

En SQLite, las bases de datos se almacenan en un solo archivo (e.g., mi_base.db), lo que las hace portátiles y fáciles de usar en Python sin necesidad de un servidor dedicado.

Categorías de Comandos SQL

SQL se divide en sublenguajes según su propósito:

- **DDL (Data Definition Language):** Define la estructura de la base de datos. Ejemplos: CREATE, ALTER, DROP.
- **DML (Data Manipulation Language):** Manipula los datos existentes. Ejemplos: INSERT, UPDATE, DELETE.
- **DQL (Data Query Language):** Consulta y recupera datos. Ejemplo principal: SELECT.
- **DCL (Data Control Language):** Gestiona permisos y transacciones. Ejemplos: GRANT, REVOKE, COMMIT (SQLite soporta transacciones básicas).
- **TCL (Transaction Control Language):** Controla transacciones para mantener la integridad. Ejemplos: BEGIN TRANSACTION, COMMIT, ROLLBACK.

15 Conexión a BDs

Transacciones

Una de las características más importantes que deben cumplir nuestras bases de datos es la consistencia de sus datos y operaciones.
Decimos que soportan ACID: Atomicidad, Consistencia, Aislamiento, Durabilidad.

ACID Explicado

- **A - Atomicidad:** Asegura que todas las operaciones dentro de una transacción se completen con éxito. Si una sola parte falla, la transacción completa es abortada y la base de datos permanece inalterada. Es un enfoque de "todo o nada" para la integridad de los datos.
- **C - Consistencia:** Garantiza que una transacción lleve la base de datos de un estado válido a otro, haciendo cumplir todas las reglas, restricciones y disparadores definidos. Esto mantiene los datos precisos y fiables en todo momento.
- **I - Aislamiento:** Se asegura de que las transacciones se procesen de forma segura e independiente al mismo tiempo, sin interferencias. Incluso si muchas transacciones ocurren simultáneamente, cada una se comporta como si fuera la única en ejecución.
- **D - Durabilidad:** Asegura que una vez que una transacción ha sido confirmada (committed), sus cambios son permanentes, incluso en caso de un fallo del sistema o un corte de energía. Los datos no se perderán.

En Python con SQLite, estos comandos se envían como strings a través de un cursor, por ejemplo:

15 Conexión a BDs

```
import sqlite3

conn = sqlite3.connect('mi_base.db')
cursor = conn.cursor()
cursor.execute("CREATE TABLE clientes (id INTEGER PRIMARY KEY, nombre
TEXT);")
conn.commit()
```

Un **cursor** en python.sqlite3 es un objeto que actúa como intermediario para interactuar con la base de datos, permitiendo ejecutar comandos SQL y recorrer los resultados de las consultas. Es una estructura de control que te permite ejecutar sentencias (SELECT, INSERT, UPDATE, DELETE), procesar los resultados fila por fila de manera eficiente y realizar operaciones sobre ellos.

Características principales:

- **Ejecución de comandos**: Se utiliza para ejecutar sentencias SQL usando el método .execute().
- **Obtención de resultados**: Permite recuperar datos del conjunto de resultados de una consulta. Los métodos comunes son fetchone() (obtiene una fila) o fetchall() (obtiene todas las filas restantes).
- **Gestión de posición**: Mantiene la posición actual dentro de un conjunto de resultados, lo que permite recuperar datos en lotes o procesarlos secuencialmente.
- **Eficiencia**: Mejora el rendimiento al no requerir la carga de todas las filas en memoria a la vez.

Comandos Básicos de SQL

Veamos ejemplos detallados de los comandos más comunes, con sintaxis estándar que funciona en SQLite.

CREATE TABLE: Crea una nueva tabla:

```
CREATE TABLE empleados (
id INTEGER PRIMARY KEY AUTOINCREMENT, -- Clave primaria autoincremental
nombre TEXT NOT NULL,   -- No permite valores nulos
salario REAL,     -- Número decimal
departamento_id INTEGER,-- Clave foránea FOREIGN KEY (departamento_id)
REFERENCES departamentos(id)  );
```

Esto define una tabla con restricciones (constraints) para integridad.

INSERT INTO: Agrega datos:

```
INSERT INTO empleados (nombre, salario, departamento_id)
VALUES ('Juan Pérez', 45000.50, 1), ('Ana López', 50000, 2), ('Carlos
Ruiz', 60000, 1);
```

SELECT: Consulta datos. Es el comando más usado:

```
SELECT * FROM empleados;
SELECT nombre, salario FROM empleados WHERE salario > 50000;
SELECT COUNT(*) AS total FROM empleados;
```

Cláusulas comunes:

- **WHERE:** Filtra (e.g., WHERE departamento_id = 1 AND salario < 60000).
- **ORDER BY:** Ordena (e.g., ORDER BY salario DESC).
- **GROUP BY:** Agrupa para funciones agregadas como SUM, AVG, MIN, MAX.
- **HAVING:** Filtra grupos (e.g., HAVING AVG(salario) > 50000).
- **LIMIT:** Limita resultados (e.g., LIMIT 10).

```
SELECT department, SUM(sales) AS total_sales
FROM sales_data
GROUP BY department
HAVING SUM(sales) > 10000;
```

UPDATE: Modifica datos

```
UPDATE empleados SET salario = salario * 1.10 WHERE departamento_id = 1;
```

DELETE: Elimina datos

```
DELETE FROM empleados WHERE id = 5;
```

JOIN: Combina tablas. SQL permite unir tablas para consultas complejas:

```
SELECT e.nombre, d.nombre AS departamento FROM empleados e
INNER JOIN departamentos d ON e.departamento_id = d.id WHERE d.nombre =
'Ventas';
```

Tipos de JOIN: INNER (coincidencias), LEFT (incluye todo de la izquierda), RIGHT, FULL.

Una explicación más detallada sobre SQL JOINs la podemos encontrar aquí.

15 Conexión a BDs

Conceptos Avanzados Introductorios

Índices: Mejoran la velocidad de consultas.

```
CREATE INDEX idx_nombre ON empleados(nombre);
```

Se crea un índice llamado **idx_nombre** sobre la tabla **empleados** que se basará en la columna **nombre**. Esto hará que las búsquedas de empleados por nombre sean más rápidas.

Vistas (Views): Tablas virtuales basadas en consultas.

```
CREATE VIEW vista_empleados AS SELECT * FROM empleados WHERE salario >
50000;
```

Subconsultas: Consultas anidadas, e.g.,

```
SELECT nombre FROM empleados WHERE salario > (SELECT AVG(salario) FROM
empleados);
```

Funciones: SQL incluye funciones como UPPER, LOWER, DATE, etc.
En SQLite: SELECT strftime('%Y-%m-%d', 'now'); para fechas. Podemos ver las funciones más importantes aquí.

Transacciones: Aseguran atomicidad. En Python: conn.begin(); ... conn.commit(); o conn.rollback();.

```
import sqlite3

# 1. Crear una conexión a la base de datos
# Si el archivo no existe, se creará automáticamente
conexion = sqlite3.connect('mi_base_de_datos.db')
cursor = conexion.cursor()

try:
    # 2. Crear una tabla si no existe
    cursor.execute('''
        CREATE TABLE IF NOT EXISTS usuarios (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            nombre TEXT NOT NULL,
            edad INTEGER
        )
    ''')
```

```
    # 3. Insertar datos (primera transacción)
    cursor.execute("INSERT INTO usuarios (nombre, edad) VALUES (?, ?)",
('Carlos', 30))
    print("Registro insertado con éxito.")

    # 4. Modificar datos (segunda transacción)
    cursor.execute("UPDATE usuarios SET edad = ? WHERE nombre = ?", (31,
'Carlos'))
    print("Edad actualizada con éxito.")

    # 5. Confirmar todos los cambios
    conexion.commit()
    print("Transacción completada y cambios guardados.")

except sqlite3.Error as e:
    # 6. Deshacer los cambios si ocurre un error
    conexion.rollback()
    print(f"Ocurrió un error: {e}. Se han deshecho los cambios.")

finally:
    # 7. Cerrar la conexión
    if conexion:
        conexion.close()
        print("Conexión cerrada.")
```

Explicación de los pasos:

- sqlite3.connect('mi_base_de_datos.db'): Conecta a la base de datos; si el archivo mi_base_de_datos.db no existe, lo crea.
- conexion.cursor(): Crea un objeto cursor, que se utiliza para ejecutar comandos SQL.
- cursor.execute(): Ejecuta una sentencia SQL. Se recomienda usar marcadores de posición (?) para insertar valores y prevenir ataques de inyección SQL.
- conexion.commit(): Guarda todos los cambios realizados desde la última confirmación. Sin esto, las inserciones y modificaciones no se guardan permanentemente.
- conexion.rollback(): Deshace todos los cambios realizados desde la última confirmación en caso de error.
- conexion.close(): Cierra la conexión a la base de datos.

15 Conexión a BDs

Relación con SQLite y Python

SQLite es un dialecto de SQL que implementa la mayoría de estos comandos, pero no necesita servidor (serverless), y está enfocado en aplicaciones locales. En Python, el módulo sqlite3 traduce comandos SQL a operaciones en disco. Por ejemplo, después de ejecutar SQL, puedes recuperar (fetch) resultados con cursor.fetchall(), para procesarlos en Python.

Ventajas de SQL en este contexto:

- **Portabilidad**: El mismo SQL funciona en otros DBMS con mínimas adaptaciones.
- **Eficiencia**: SQLite es rápido para bases de datos pequeñas (hasta GBs).
- **Seguridad**: Evita inyecciones SQL usando marcadores (placeholders): cursor.execute("SELECT * FROM tabla WHERE id = ?", (valor,)).

Buenas Prácticas y Consideraciones

- **Normalización**: Diseña tablas para evitar redundancias (e.g., formas normales 1NF, 2NF, 3NF).
- **Errores Comunes:** Olvidar COMMIT (aceptar) en transacciones, o no manejar errores (usa try-except en Python).
- **Aprendizaje Práctico:** Prueba comandos en herramientas como DB Browser for SQLite o directamente en Python.
- **Limitaciones de SQLite:** No soporta RIGHT JOIN nativo (usa LEFT y swaps), ni usuarios/roles.

Uso de SQLite con Python y DB Browser for SQLite

Introducción a SQLite

SQLite es un sistema de gestión de bases de datos relacionales (RDBMS) ligero, de código abierto y sin servidor. A diferencia de otros RDBMS como MySQL o PostgreSQL, SQLite no requiere un proceso de servidor separado, ya que almacena toda la base de datos en un solo archivo en disco.

Características de SQLite

- **Ligero**: Ocupa poco espacio (menos de 750 KB con todas las funciones habilitadas).
- **Sin dependencias externas:** Todo está incluido en el módulo SQLite de Python.

15 Conexión a BDs

- **Sin servidor**: No necesita configuración ni administración de servidor.
- Base de datos almacenada en **un solo archivo.**
- Portable: Compatible con múltiples plataformas (Windows, macOS, Linux).
- **Transaccional**: Soporta transacciones ACID (Atomicidad, Consistencia, Aislamiento, Durabilidad).
- **Ampliamente utilizado**: Está integrado en dispositivos móviles, navegadores y aplicaciones de escritorio.

¿Cuándo usar SQLite?

- Aplicaciones pequeñas o medianas.
- Almacenamiento local en aplicaciones móviles o de escritorio.
- Prototipos que luego pueden migrar a bases de datos más robustas.
- Proyectos que requieren simplicidad y no alta concurrencia.

Limitaciones

- No soporta ciertas operaciones avanzadas como RIGHT OUTER JOIN o FULL OUTER JOIN.
- No es ideal para aplicaciones con alta concurrencia o grandes volúmenes de datos.

Uso de SQLite con Python

Python incluye el módulo sqlite3, que proporciona una interfaz compatible con la especificación DB-API 2.0 (PEP 249) que es es un estándar de Python para el acceso a bases de datos relacionales, que define una interfaz común y consistente para interactuar con diferentes motores de bases de datos. Proporciona una serie de clases y métodos estándar para objetos como Connection y Cursor, permitiendo escribir código portable entre distintas bases de datos y lenguajes.

Este módulo está integrado en Python desde la versión 2.5, por lo que no requiere instalación adicional.

Podemos ver la documentación completa aquí.

Instalación y configuración

Para verificar que el módulo sqlite3 está disponible, ejecuta:

```
import sqlite3
```

Si no hay errores, estás listo para comenzar.

15 Conexión a BDs

Crear una base de datos

Para crear una base de datos SQLite, usa la función sqlite3.connect(). Si el archivo especificado no existe, SQLite lo creará automáticamente.

Ejemplo: Crear una base de datos:

```
import sqlite3

# Conectar a la base de datos (creará el archivo si no existe)
conn = sqlite3.connect('mi_base_de_datos.db')
# Cerrar la conexión
conn.close()
```

Crear tablas

Usa el comando SQL CREATE TABLE para definir la estructura de una tabla. Es importante especificar los tipos de datos de las columnas (por ejemplo, INTEGER, TEXT, REAL).

Ejemplo:

```
import sqlite3

conn = sqlite3.connect('mi_base_de_datos.db')
cursor = conn.cursor()

# Crear una tabla
cursor.execute('''CREATE TABLE IF NOT EXISTS usuarios ( id INTEGER
PRIMARY KEY,nombre TEXT NOT NULL, edad INTEGER, email TEXT UNIQUE) ''')

# Guardar los cambios
conn.commit()

# Cerrar la conexión
conn.close()
```

Insertar datos

Usa el comando SQL INSERT INTO para agregar registros. Para evitar inyecciones SQL (uso malintencionado de los datos para corromper el sistema), utiliza marcadores de posición (?) y pasa los valores como una tupla.

15 Conexión a BDs

Ejemplo:

```
import sqlite3

conn = sqlite3.connect('mi_base_de_datos.db')
cursor = conn.cursor()

# Insertar un registro
cursor.execute("INSERT INTO usuarios (nombre, edad, email) VALUES (?, ?,
?)", ('Juan Pérez', 30, 'juan@example.com'))

# Guardar los cambios
conn.commit()
# Cerrar la conexión
conn.close()
```

Consultar datos

Usa el comando SQL SELECT para obtener datos. Puedes usar métodos como fetchall(), fetchone() o fetchmany() para recuperar los resultados, según el número de resultados quieras recuperar.

Ejemplo:

```
import sqlite3

conn = sqlite3.connect('mi_base_de_datos.db')
cursor = conn.cursor()

# Consultar todos los registros
cursor.execute("SELECT * FROM usuarios") resultados = cursor.fetchall()

# Imprimir los resultados
for fila in resultados:
print(fila)

# Cerrar la conexión
conn.close()
```

Si queremos filtrar datos, podemos añadir una condición con WHERE. Por ejemplo para obtener sólo los usuarios con edad superior a 25 años:

```
cursor.execute('SELECT nombre FROM usuarios WHERE edad > ?', (25,))
resultados = cursor.fetchall()
```

15 Conexión a BDs

Actualizar datos

Usa el comando SQL UPDATE para modificar registros existentes.
Ejemplo: Actualizar la edad de un usuario:

```
import sqlite3

conn = sqlite3.connect('mi_base_de_datos.db') cursor = conn.cursor()

# Actualizar un registro
cursor.execute("UPDATE usuarios SET edad = ? WHERE nombre = ?", (31,
'Juan Pérez'))

# Guardar los cambios
conn.commit()
# Cerrar la conexión
conn.close()
```

Eliminar datos

Usa el comando SQL DELETE FROM para eliminar registros.

Ejemplo:

```
import sqlite3

conn = sqlite3.connect('mi_base_de_datos.db')
cursor = conn.cursor()

# Eliminar un registro

cursor.execute("DELETE FROM usuarios WHERE nombre = ?", ('Juan Pérez',))

# Guardar los cambios
conn.commit()
# Cerrar la conexión
conn.close()
```

Manejo de transacciones

SQLite soporta transacciones que agrupan operaciones para garantizar la integridad de los datos, suelen ser varios pasos que o se hacen todos, o

no se hace ninguno. Un ejemplo típico de transacción sería, el registro de una transferencia bancaria: el dinero se saca de una cuenta y se ingresa en la otra, si cualquiera de las dos falla, no se completa. Usa commit() para confirmar cambios, y rollback(), para deshacerlos en caso de error.

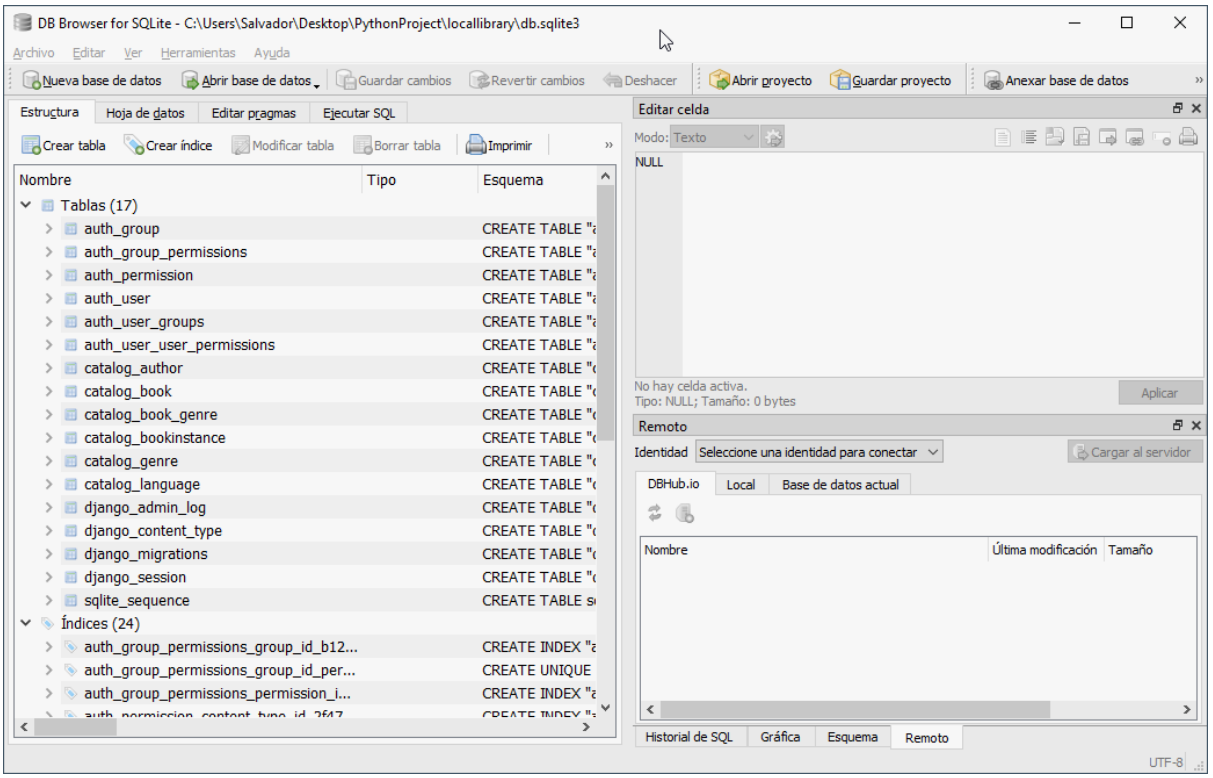
Ejemplo:

```
import sqlite3

conn = sqlite3.connect('mi_base_de_datos.db')
cursor = conn.cursor()

try:
      # Iniciar transacción
      conn.execute('BEGIN')

      # Insertar registros
      sql = "INSERT INTO customers (name, address) VALUES (%s, %s)"
      val = ("John", "Highway 21")
      cursor.execute(sql, val)

      # Confirmar transacción
      conn.commit()

except sqlite3.Error as e:

      # Revertir transacción en caso de error
      conn.rollback()
      print(f"Error: {e}")

# Cerrar la conexión
conn.close()
```

Uso de DB Browser for SQLite

DB Browser for SQLite (anteriormente SQLiteBrowser), es una herramienta gráfica de código abierto que permite crear, diseñar y editar bases de datos SQLite. Su interfaz similar a una hoja de cálculo, facilita la gestión de datos sin necesidad de escribir comandos SQL, aunque también soporta consultas SQL.

15 Conexión a BDs



Instalación

Descarga la versión más reciente desde https://sqlitebrowser.org/

Ejemplo 1: Base de datos de películas

Este ejemplo crea una base de datos de películas, inserta datos con Python, y los visualiza con DB Browser for SQLite:

Código Python Ejemplo 1

```
import sqlite3

# Conectar a la base de datos
conn = sqlite3.connect('peliculas.db')
cursor = conn.cursor()

# Crear tabla
cursor.execute('''CREATE TABLE IF NOT EXISTS peliculas ( id INTEGER
PRIMARY KEY, titulo TEXT NOT NULL, anio INTEGER, director TEXT) ''')

# Insertar datos
cursor.execute("INSERT INTO peliculas (titulo, anio, director) VALUES
(?, ?, ?)",('El Padrino', 1972, 'Francis Ford Coppola'))
```

```
cursor.execute("INSERT INTO peliculas (titulo, anio, director) VALUES
(?, ?, ?)",('Pulp Fiction', 1994, 'Quentin Tarantino'))

# Guardar cambios
conn.commit()
# Cerrar conexión
conn.close()
```

En DB Browser for SQLite

- Abre peliculas.db.
- Ve a la pestaña "Examinar datos" y selecciona la tabla de películas.
- Verifica que los datos insertados estén presentes.
- Agrega un nuevo registro manualmente (por ejemplo, Título: Inception, Año: 2010, Director: Christopher Nolan).

Ejemplo 2: Consultar y filtrar datos

Consulta películas del año 1994, usando Python y DB Browser for SQLite:

Código Python

```
import sqlite3

conn = sqlite3.connect('peliculas.db')
cursor = conn.cursor()

# Consultar películas del año 1994
cursor.execute("SELECT * FROM peliculas WHERE anio = ?", (1994,))
resultados = cursor.fetchall()

# Imprimir resultados
for fila in resultados:
      print(fila)

# Cerrar conexión
conn.close()
```

En DB Browser for SQLite

- Abre peliculas.db.
- Ve a la pestaña "Ejecutar SQL".
- Escribe: SELECT * FROM peliculas WHERE anio = 1994.
- Haz clic en "Ejecutar" para ver los resultados.

15 Conexión a BDs

Ejemplo 3: Gestión básica de usuarios (CRUD elemental)

Este ejemplo básico incluye las sentencias SQL para crear una tabla, insertar datos, consultar, actualizar y eliminar un usuario. Es ideal para aprender las operaciones básicas, a las que se denomina **CRUD** (**Create**, **Read**, **Update**, **Delete**).

Por supuesto faltaría la lógica de la aplicación:

Código Ejemplo 3

```
import sqlite3

# Conectar o crear base de datos
conn = sqlite3.connect('usuarios.db')
cursor = conn.cursor()

# Crear tabla usuarios si no existe
cursor.execute(''' CREATE TABLE IF NOT EXISTS usuarios ( id INTEGER
PRIMARY KEY AUTOINCREMENT, nombre TEXT NOT NULL, email TEXT UNIQUE NOT
NULL, edad INTEGER ) ''')

# Insertar un usuario
cursor.execute('INSERT INTO usuarios (nombre, email, edad) VALUES (?, ?,
?)', ("Carlos", "carlos@example.com", 30))
conn.commit()

# Consultar todos los usuarios
cursor.execute('SELECT * FROM usuarios')
print("Usuarios en la base de datos:")
for fila in cursor.fetchall():
      print(fila)

# Actualizar edad de un usuario
cursor.execute('UPDATE usuarios SET edad = ? WHERE nombre = ?', (31,
"Carlos"))
conn.commit()

# Eliminar usuario por nombre
cursor.execute('DELETE FROM usuarios WHERE nombre = ?', ("Carlos",))
conn.commit()

# Cerrar conexión
conn.close()
```

15 Conexión a BDs

Conceptos utilizados: creación de tabla, parámetros con ? para evitar inyección SQL, operaciones CRUD básicas.

Ejemplo 4: Aplicación sencilla de tareas (To-Do List) con marcado de completado

Creamos una tabla tareas, con campos para descripción y estado de completado. Implementamos funciones para añadir, listar, marcar tareas completadas y eliminar.

Faltaría la lógica de la aplicación, para conectar las funciones con los menús o el interfaz:

Código del Ejemplo 4

```
import sqlite3

def crear_conexion():
      return sqlite3.connect('tareas.db')

def crear_tabla(conn):
      cursor = conn.cursor()
      cursor.execute('''CREATE TABLE IF NOT EXISTS tareas (id INTEGER
PRIMARY KEY AUTOINCREMENT, descripcion TEXT NOT NULL, completada INTEGER
NOT NULL DEFAULT 0 CHECK(completada IN (0,1))) ''')
      conn.commit()

def agregar_tarea(conn, descripcion):
      cursor = conn.cursor()
      cursor.execute('INSERT INTO tareas (descripcion) VALUES (?)',
(descripcion,))
      conn.commit()

def listar_tareas(conn):
      cursor = conn.cursor()
      cursor.execute('SELECT id, descripcion, completada FROM tareas')
      return cursor.fetchall()

def marcar_como_completada(conn, id_tarea):
      cursor = conn.cursor()
      cursor.execute('UPDATE tareas SET completada = 1 WHERE id = ?',
(id_tarea,))
      conn.commit()

def eliminar_tarea(conn, id_tarea):
```

```
    cursor = conn.cursor()
    cursor.execute('DELETE FROM tareas WHERE id = ?', (id_tarea,))

# Uso
conn.commit()
conn = crear_conexion()
crear_tabla(conn)

agregar_tarea(conn, "Aprender SQLite")
agregar_tarea(conn, "Practicar Python")

print("Tareas:")
for tarea in listar_tareas(conn):
    print(tarea)

marcar_como_completada(conn, 1)
eliminar_tarea(conn, 2)

print("\nTareas después de actualizar:")
for tarea in listar_tareas(conn):
    print(tarea) conn.close()
```

Conceptos: validación con CHECK para campo booleano, funciones reutilizables para operaciones, lógica de estado para tareas...

Ejemplo 5: Sistema avanzado con búsquedas y múltiples filtros

Este ejemplo crea una tabla con información más compleja, añade múltiples registros en una única sentencia y realiza consultas con filtros usando parámetros, ordenamientos y búsquedas por texto.

Código del Ejemplo 5

```
import sqlite3

conn = sqlite3.connect('productos.db')
cursor = conn.cursor()

# Crear tabla productos
cursor.execute('''CREATE TABLE IF NOT EXISTS productos ( id INTEGER
PRIMARY KEY AUTOINCREMENT, nombre TEXT NOT NULL, categoria TEXT, precio
REAL CHECK(precio >= 0), stock INTEGER CHECK(stock >= 0))''')
```

15 Conexión a BDs

```
# Insertar varios productos
productos = [
("Camiseta", "Ropa", 15.99, 50),
("Pantalones", "Ropa", 39.99, 20),
("Ratón", "Electrónica", 25.75, 15),
("Teclado", "Electrónica", 45.40, 10),
("Libro Python", "Libros", 29.95, 100)
]

cursor.executemany('INSERT INTO productos (nombre, categoria, precio,
stock) VALUES (?, ?, ?, ?)', productos)
conn.commit()

# Consultar productos en categoría "Electrónica" con stock > 5,
ordenados por precio descendente
categoria = "Electrónica"
stock_min = 5
cursor.execute('''SELECT nombre, precio, stock FROM productos WHERE
categoria = ? AND stock > ? ORDER BY precio DESC''', (categoria,
stock_min))
resultados = cursor.fetchall()

print(f"Productos en categoría '{categoria}' con stock > {stock_min}:")
for producto in resultados:
      print(producto)

# Buscar productos cuyo nombre contenga 'Libro'
busqueda = "%Libro%"
cursor.execute('SELECT nombre, categoria, precio FROM productos WHERE
nombre LIKE ?', (busqueda,))
productos_encontrados = cursor.fetchall()

print("\nProductos que contienen 'Libro' en el nombre:")
for producto in productos_encontrados:
      print(producto)

conn.close()
```

Conceptos: uso de executemany para insertar varios registros, restricciones de integridad con CHECK, consultas con filtros complejos, ordenamientos, y búsqueda con LIKE para texto.

15 Conexión a BDs

Conclusiones y buenas prácticas

- **Aprende SQL**: Es fundamental para realizar consultas avanzadas y optimizar el uso de bases de datos.
- **Usa SQLite para proyectos pequeños**: Es ideal para aplicaciones locales o prototipos.
- **Aprovecha DB Browser for SQLite**: Facilita la gestión visual de bases de datos.
- **Evita inyecciones SQL**: Usa marcadores de posición (?) en consultas.
- **Cierra conexiones**: Libera recursos cerrando conexiones con conn.close().
- **Usa transacciones**: Garantiza la integridad de los datos en operaciones complejas.

Tabla de tipos de datos SQLite y equivalentes en Python

Vamos a ver los diferentes tipos de datos que se pueden usar en SQLite, y su correspondencia con los datos de Python:

Tipo de datos SQLite	Tipo datos	Descripción
NULL	None	Representa un valor nulo
INTEGER	int	Números enteros
REAL	float	Números de punto flotante
TEXT	str	Cadenas texto (UTF-8, UTF-16BE, UTF-16LE)
BLOB	bytes	Datos binarios (imágenes, archivos, etc.)

15 Conexión a BDs

Conexión a una Base de datos MySQL

Para conectar Python con MySQL, primero debes instalar el paquete mysql-connector-python usando pip: pip install mysql-connector-python. Luego, en tu script de Python, importa el conector y usa la función mysql.connector.connect() para establecer la conexión, especificando los parámetros host, user, password y database.

1. Instala el conector de MySQL

Abre tu terminal o consola y ejecuta el siguiente comando para instalar el paquete necesario:

```
pip install mysql-connector-python
```

2. Escribe el código Python

Crea un archivo de Python e importa la biblioteca mysql.connector. Luego, usa mysql.connector.connect() con tus credenciales de base de datos para establecer una conexión.

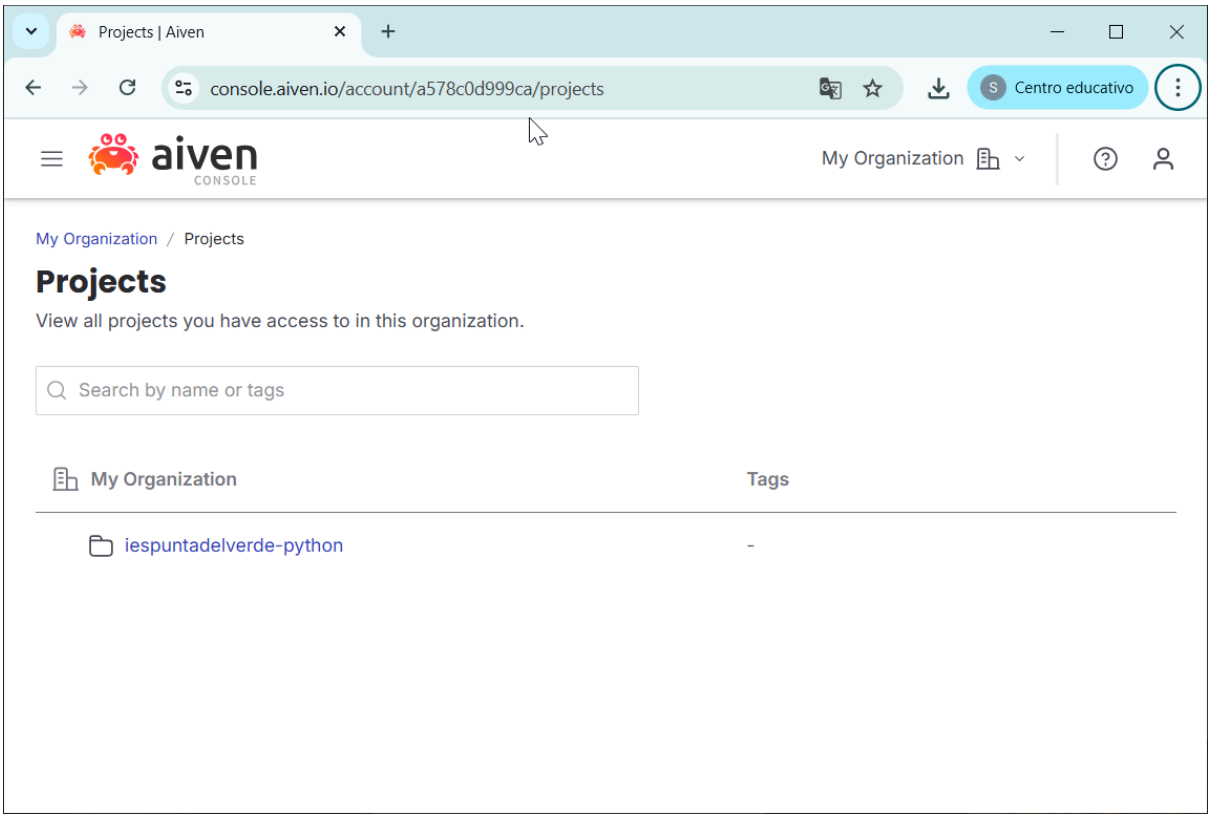
```
import mysql.connector

try:
    # Reemplaza con tus datos de conexión
    conexion = mysql.connector.connect(
        host="localhost",        # O la IP de tu servidor
        user="tu_usuario",       # Tu nombre de usuario de MySQL
        password="tu_contraseña", # Tu contraseña
        database="nombre_de_tu_bd" # El nombre de la base de datos a la
que te quieres conectar
    )
    print("¡Conexión exitosa a MySQL!")

    # Aquí puedes ejecutar consultas SQL
    # Ejemplo:
    # cursor = conexion.cursor()
    # cursor.execute("SELECT * FROM tu_tabla")
    # resultados = cursor.fetchall()
    # for fila in resultados:
    #     print(fila)
```

```
except mysql.connector.Error as err:
    print(f"Error al conectar: {err}")

finally:
    if 'conexion' in locals() and conexion.is_connected():
        conexion.close()
        print("Conexión a MySQL cerrada.")
```

Para hacer una prueba de conexión con una base de datos MySQL podemos descargarnos XAMPP e instalarlo en local en nuestro equipo.

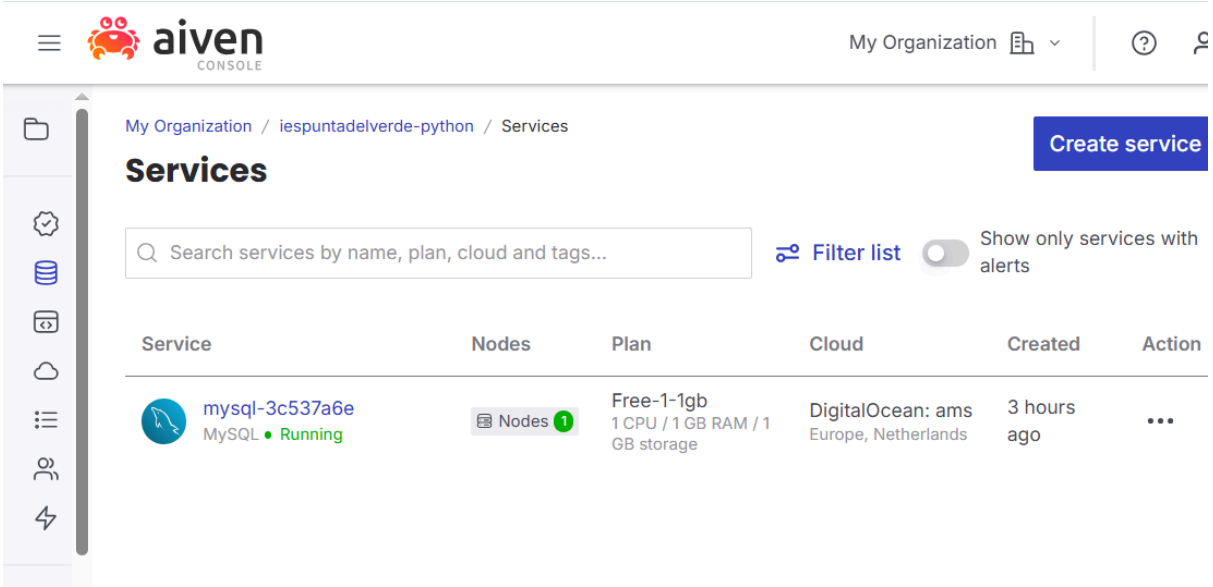
MySQL en la nube

También podemos usar un servicio gratuito en la nube como Aiven

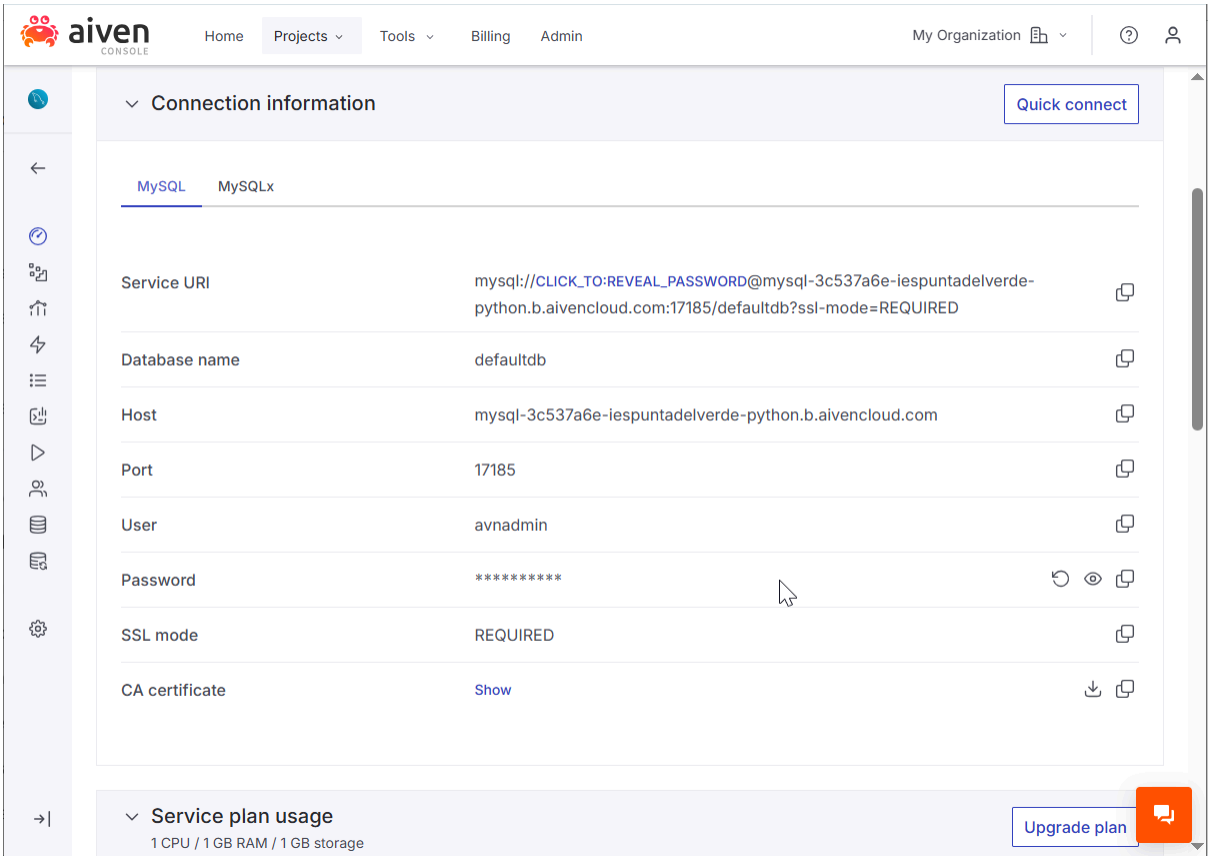


Podemos configurar una Base de Datos y conectar con ella:

15 Conexión a BDs

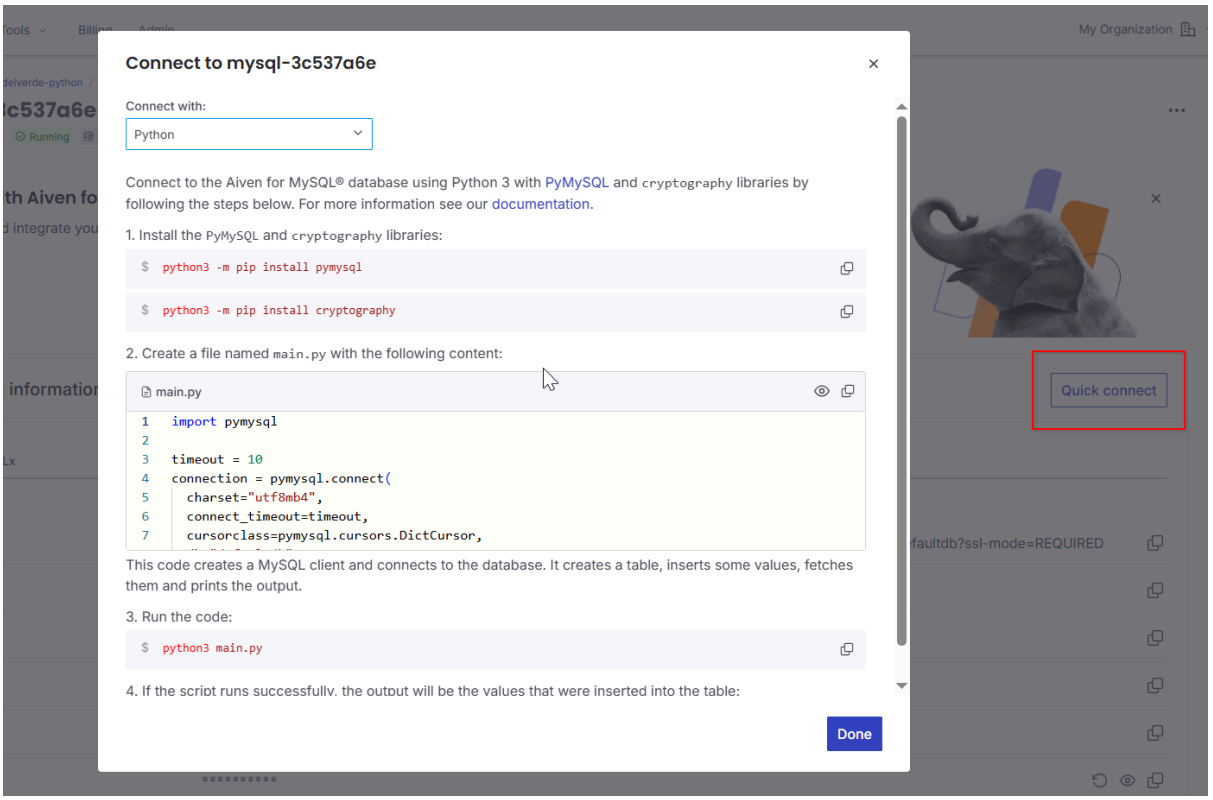


Aiven te facilita la información sobre la conexión:



Aiven te facilita el código para conectar a la BD con Python:

15 Conexión a BDs



El código para conectar sería el siguiente:

```
import pymysql

timeout = 10
connection = pymysql.connect(
    charset="utf8mb4",
    connect_timeout=timeout,
    cursorclass=pymysql.cursors.DictCursor,
    db="defaultdb",
    host="mysql-3c537a6e-iespuntadelverde-python.b.aivencloud.com",
    password="************",
    read_timeout=timeout,
    port=17185,
    user="avnadmin",
    write_timeout=timeout,
)

try:
    cursor = connection.cursor()
    cursor.execute("DROP TABLE IF EXISTS mytest")
    cursor.execute("CREATE TABLE mytest (id INTEGER PRIMARY KEY)")
    cursor.execute("INSERT INTO mytest (id) VALUES (1), (2)")
    cursor.execute("SELECT * FROM mytest")
    print(cursor.fetchall())
```

15 Conexión a BDs

```
finally:
    connection.close()
```

Debería devolver:

```
[{'id': 1}, {'id': 2}]
```